Resumen

Este informe presenta el diseño, implementación, despliegue y validación de GroupsApp, una aplicación de mensajería distribuida orientada principalmente a comunicación grupal, con soporte adicional para mensajería directa entre usuarios. La solución fue construida bajo una arquitectura de microservicios con NestJS como base de servicios, REST como interfaz pública, gRPC para comunicación interna síncrona, RabbitMQ para integración asíncrona, PostgreSQL como persistencia transaccional principal, Redis para estado efímero de presencia, Amazon S3 para adjuntos, y Amazon EKS como plataforma de orquestación sobre AWS.

Arquitectura final desplegada

La solución quedó compuesta por:

Componente	Responsabilidad principal	Tecnología
Frontend	Interfaz de demostración, consumo de API pública	Nginx + JS estático
api-gateway	Fachada REST, validación JWT, orquestación gRPC	NestJS
auth-service	Registro, login, emisión/validación de JWT	NestJS + PostgreSQL
group-service	Grupos, miembros, admins, canales	NestJS + PostgreSQL
user-service	Contactos, bloqueos, presencia, política de DM	NestJS + PostgreSQL + Redis
message-service	Mensajes, receipts, adjuntos, outbox	NestJS + PostgreSQL + S3 + RabbitMQ
notification-service	Consumo idempotente de eventos	NestJS + RabbitMQ + PostgreSQL
RabbitMQ	Integración asíncrona, retry, DLQ	RabbitMQ
Redis	Estado efímero de presencia	Redis
PostgreSQL RDS	Persistencia transaccional	Amazon RDS PostgreSQL
S3	Archivos y adjuntos	Amazon S3

Componente	Responsabilidad principal	Tecnología
EKS	Orquestación y escalado de contenedores	Amazon EKS
ECR	Registro de imágenes	Amazon ECR
ALB Ingress	Exposición pública y enrutamiento	AWS ALB

Microservicios implementados

api-gateway

Expone la API REST pública bajo el prefijo /api/v1. Su responsabilidad principal es servir como fachada unificada para el cliente, validar autenticación por JWT y delegar operaciones hacia los servicios internos mediante gRPC.

auth-service

Gestiona:

- registro de usuarios,
- hash de contraseñas,
- autenticación,
- emisión y validación de JWT.

group-service

Gestiona:

- creación de grupos,
- membresías,
- administradores,
- canales/subgrupos.

user-service

Gestiona:

- contactos,
- bloqueos,
- política de mensajería directa,
- presencia online/offline usando Redis.

message-service

Gestiona:

- envío y lectura de mensajes,
- mensajes en canales,
- mensajes directos,
- receipts delivered/read,
- adjuntos con S3,
- outbox persistente y relay de eventos.

notification-service

Consume eventos `message.created` desde RabbitMQ. Implementa:

- deduplicación,
- idempotencia,
- consumo al menos una vez,
- soporte para reintentos y DLQ básicos.

frontend

Interfaz funcional para la demostración final, desplegada como frontend separado, servido por Nginx, usando el mismo ALB que el backend y consumiendo `/api/v1/...`.

Modelo de datos distribuido

Estrategia

Se adoptó el patrón base de datos por servicio con separación lógica por contexto de negocio. Esto reduce acoplamiento entre dominios, facilita evolución independiente y permite alinear cada esquema con la semántica de cada servicio.

Base de datos	Servicio/contexto	Contenido principal
groupsapp_auth	auth-service	usuarios, credenciales, autenticación
groupsapp_group	group-service	grupos, miembros, admins, canales
groupsapp_user	user-service	contactos, bloqueos
groupsapp_message	message-service	mensajes, conversaciones, receipts, adjuntos, outbox

Redis

Se utilizó Redis para presencia mediante claves del tipo:

presence:user:{user_id}

con TTL asociado. Este estado es efímero y no requiere persistencia relacional.

S3

Los adjuntos se almacenan en S3, usando una organización lógica por:

Propietario/attachment_id/nombre de archivo.

Particionamiento, replicación, consistencia y transacciones

Particionamiento

El particionamiento físico de msg.messages se implementó en PostgreSQL por shard_id (HASH). El shard_id se calcula en la aplicación a partir de channel_id, por lo que funcionalmente la distribución de mensajes está orientada por canal: todos los mensajes de un mismo canal caen en la misma partición.

Las operaciones más frecuentes (listar historial, últimos N mensajes, continuar chat) siempre filtran por canal. Al caer todos los mensajes del mismo canal en el mismo shard/partición esto mejora la localidad de datos e índices, reduce lecturas dispersas entre particiones, simplifica el manejo de escaneos por canal.

Pero esto representa un trade-off, ya que se pueden presentar hot channels que generen desbalance entre particiones. A pesar de esto, se considera mayor el beneficio obtenido en las lecturas y escrituras por conversación, siendo esto el foco de la aplicación.

Replicación

La capa relacional principal quedó desplegada en Amazon RDS PostgreSQL con Multi-AZ habilitado.

Esto aporta:

- redundancia administrada
- réplica standby en otra zona de disponibilidad
- failover automático
- mayor disponibilidad.

Comunicaciones del sistema

REST

La comunicación externa se realiza por REST hacia la api-gateway.

8.2 gRPC

La comunicación gRPC se usa entre api-gateway y servicios internos, y también entre microservicios de dominio cuando aplica (message-service con group-service y user-service)

8.3 RabbitMQ

La comunicación asíncrona usa RabbitMQ:

- productor: message-service
- consumidor: notification-service

Presencia y estado efímero

La presencia automática se implementó con:

- heartbeat del frontend: 10 s
- polling de presencia: 5 s
- TTL en Redis: 25 s

Esto implica que un usuario puede tardar aproximadamente hasta 30 s en aparecer como offline en el peor caso, lo cual representa un compromiso razonable entre frescura del estado y costo de actualización.

Adjuntos y archivos en S3

Flujo implementado:

1. POST /api/v1/files/upload-url
2. El backend genera una URL prefirmada
3. El cliente hace PUT directo a S3
4. POST /api/v1/files/{attachmentId}/complete
5. POST api/v1/messages/... referenciando attachment_ids

Ventajas

- el archivo no atraviesa el backend como stream principal,
- se reduce carga en api-gateway y message-service,
- S3 asume el almacenamiento escalable.

Uso de RabbitMQ

En la arquitectura distribuida RabbitMQ se utiliza como middleware de mensajería para desacoplar la publicación y el consumo de eventos de negocio, especialmente en el flujo de mensajería.

Su objetivo principal es evitar dependencias síncronas entre servicios para tareas no críticas en tiempo real.

Productor: message-service

Consumidor: notification-service

Evento principal: message.created

Flujo implementado

1. message-service persiste el mensaje en base de datos.
2. En la misma transacción registra el evento en msg.outbox (patrón Outbox).
3. Un relay publica el evento desde outbox hacia RabbitMQ.
4. notification-service consume el evento y registra su procesamiento.
5. Si ocurre fallo de consumo, se aplican reintentos, si persiste, el mensaje pasa a DLQ.

Garantías y tolerancia a fallos

Entrega: modelo at least once.

Idempotencia del consumidor: se evita reprocesamiento mediante notif.processed_events.

Reintentos automáticos: cola de retry con TTL.

DLQ: aislamiento de mensajes que no pudieron procesarse tras el máximo de reintentos.

Despliegue en AWS

Se creó una VPC propia con 6 subredes:

- 2 públicas
- 2 privadas-app
- 2 privadas-db

distribuidas en 2 zonas de disponibilidad.

Uso por capa:

- subredes públicas: ALB
- subredes privadas-app: EKS
- subredes privadas-db: RDS

Escalabilidad y alta disponibilidad (HPA)

La solución implementa escalamiento horizontal automático en Kubernetes mediante HorizontalPodAutoscaler usando la API autoscaling/v2, configurado en el overlay de AWS.

Configuración aplicada

api-gateway-hpa

- minReplicas: 2
- maxReplicas: 6
- métrica: CPU
- umbral objetivo: averageUtilization: 70%

message-service-hpa

- minReplicas: 1
- maxReplicas: 4
- métrica: CPU
- umbral objetivo: averageUtilization: 70%

El api-gateway actúa como punto de entrada del sistema y concentra todo el tráfico REST proveniente de clientes externos. Por esta razón, se definió minReplicas = 2, con el fin de mejorar la continuidad operativa y ofrecer mayor tolerancia a fallos de pod o de nodo.

Por su parte, message-service presenta una carga naturalmente variable, dependiente de la actividad de mensajería del sistema. El uso de HPA en este servicio permite incrementar el número de réplicas cuando la utilización de CPU supera el umbral configurado, y reducirlas cuando la demanda disminuye, optimizando así la relación entre costo y rendimiento.

La presencia de HPA aporta dos beneficios principales:

- escalabilidad horizontal automática al adaptar la cantidad de pods a la carga real
- mejora de disponibilidad al mantener más de una réplica en componentes críticos como api-gateway.

Esto fortalece la operación del sistema en ambiente distribuido y alinea el despliegue con los principios de elasticidad y alta disponibilidad.

Pruebas y validación

Estrategia de pruebas

Se aplicó una estrategia de validación por capas, orientada a comprobar tanto la funcionalidad del sistema como la integración entre componentes distribuidos. La estrategia incluyó:

- Pruebas funcionales end-to-end vía API REST.
- Pruebas de integración entre microservicios usando REST, gRPC y RabbitMQ.
- Pruebas de resiliencia ante fallos controlados de componentes.
- Prueba básica de desempeño y carga.

Pruebas ejecutadas

`scripts/smoke-test.sh`
Valida el flujo base del sistema: registro y login de usuarios, creación de grupo, gestión de membresía, envío de mensajes y consulta del historial.

`scripts/integration-test.sh`
Valida reglas de negocio e integración distribuida: contactos, bloqueos, política de mensajería directa, idempotencia, canales, remoción de miembros y funcionamiento del pipeline outbox-consumer.

`scripts/file-attachment-test.sh`
Valida el flujo completo de adjuntos con Amazon S3: solicitud de URL prefirmada, carga directa con PUT, confirmación de carga y envío de mensaje con attachment_ids.

`scripts/dlq-demo.sh`
Valida el comportamiento de reintentos y Dead Letter Queue (DLQ) en RabbitMQ.

`scripts/metrics-snapshot.sh`
Genera evidencia operativa del estado del sistema, incluyendo contenedores, colas RabbitMQ, outbox y consumidor.

Resiliencia por reinicio de servicio crítico
Reiniciar message-service durante una sesión activa y verificar recuperación sin pérdida visible de funcionalidad.

Resiliencia del consumidor
Detener notification-service, generar eventos pendientes, volver a levantarlo y comprobar el consumo posterior de mensajes acumulados.

Resultado general

Los resultados obtenidos muestran que la solución implementada cumple los flujos funcionales principales y los requerimientos no funcionales clave de sistemas distribuidos. En particular, se validó la integración consistente de comunicación síncrona (REST y gRPC) y asíncrona (RabbitMQ), junto con mecanismos de resiliencia básica, operación estable sobre Kubernetes y evidencia de particionamiento físico en la capa de mensajes. En conjunto, estos resultados respaldan la solidez técnica y la viabilidad del despliegue realizado en AWS.